



PROJECT OVERVIEW

Geeta Param Seva

A moderated community app. Anyone can apply to join; admins approve membership, place the roster, and issue group join PINs.

Expo SDK 57 · Expo Router · React Native · Firebase

Cloud Functions (asia-south1) · Remote Config · AdMob banners (NPA)

- Moderated community
- Group seva feed
- Adhyay & Aarti practice
- Gita chapters & verses
- English + Hindi

VERSION

1.0.1

RUNTIME

Android & iOS
Expo + EAS Build

LEGAL

Proprietary
All rights reserved

CONTACT

Sparsh Agrawal
geetaparamseva@gmail.com

Contents

01	Overview	The moderated-community model	3
02	Features	Capabilities, member and admin experiences	4
03	Architecture	Client, Firebase, native binaries	5
04	Repository map	Where each responsibility lives	6
05	Auth & roles	Applicants, members, admins, seniors	7
06	Tech stack		8
07	Setup	Prerequisites, install, secrets, seed data	9
08	Development		10
09	Backend deploy		11
10	Banner ads	Placement and Remote Config	11
11	Legal	GitHub Pages	12
12	Builds & license		13

Proprietary software. All rights reserved. You may not use, copy, modify, or distribute this codebase without prior written permission.

Support: geetaparamseva@gmail.com · Licensing: Sparshagrawaln@gmail.com

About this export

Source	README.md at the repository root
Scope	Overview through License — every section of the README
Verified	Commands, paths, and URLs checked against the repository

What this is

Geeta Param Seva is a **moderated community**: anyone may download the app and submit a join application with a name and phone number.

Community features require admin approval, roster placement, and sign-in with phone plus PIN.

Built with Expo Router on the Expo SDK, React Native, and Firebase — Auth, Firestore, Cloud Functions, and Remote Config — plus optional non-personalized AdMob banners.

Roles and surfaces

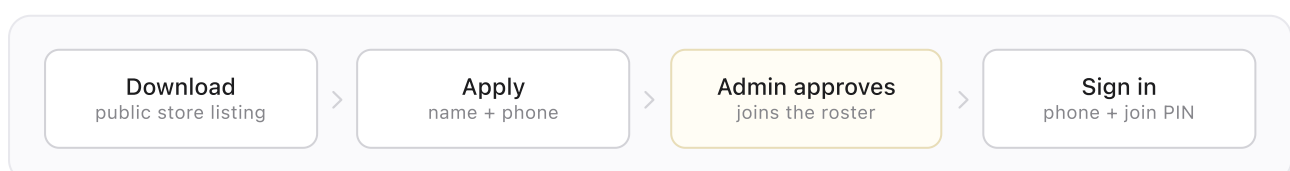
● Members	● Admins	● Seniors	● Platform
• Apply to join • Group feed • Practice today • Gita / Aarti • Push alerts	• Join applications • Roster + join PINs • Seva posts • Practice assign • Group tools	• All groups • Admin roster • Global oversight	• Expo app • Firebase • AdMob banners (NPA)

Sign-in is the boundary: applicants stay signed out, members land on the home and seva tabs, admins and seniors land on the admin dashboard.

Ground rules

● Gated community	● No personal data in git	● Non-personalized ads only
The app is publicly downloadable, but every community feature stays behind admin approval. The listing is open; the group is not.	Real senior phone numbers and roster PII live only in gitignored local files, copied from the committed <code>*.example.json</code> templates.	Banners only — no interstitial, no video, no ATT or IDFA — with a Remote Config kill switch that removes them without a release.

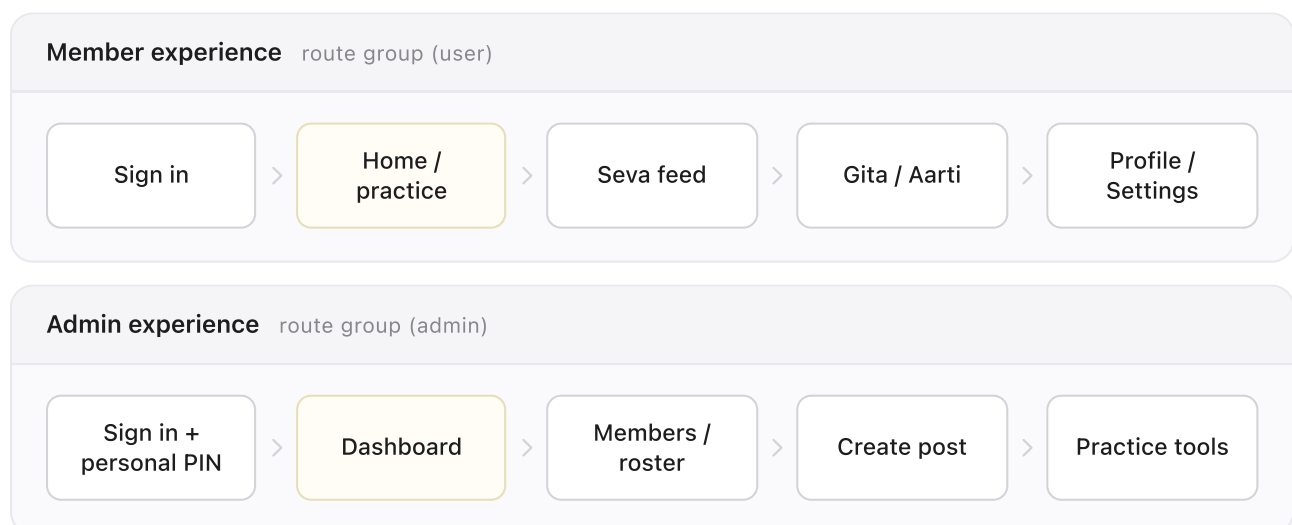
From download to member



What you get

AREA	WHAT YOU GET
Auth	Apply to join (name + phone); phone + group join PIN (members); personal PIN for admins after first login; sole-device session
Feed	Seva posts, messages, and alerts scoped to groups
Practice	Standing Adhyay / Aarti assignments with completion tracking
Scripture	In-app Gita chapters and verses
i18n	English + Hindi (with optional machine-assisted strings)
Ads	Member-only non-personalized banner ads (Remote Config kill switch); never on reading / auth screens; no ATT / IDFA tracking
Admin	Join applications, roster, join PIN rotation, content publishing, practice overview

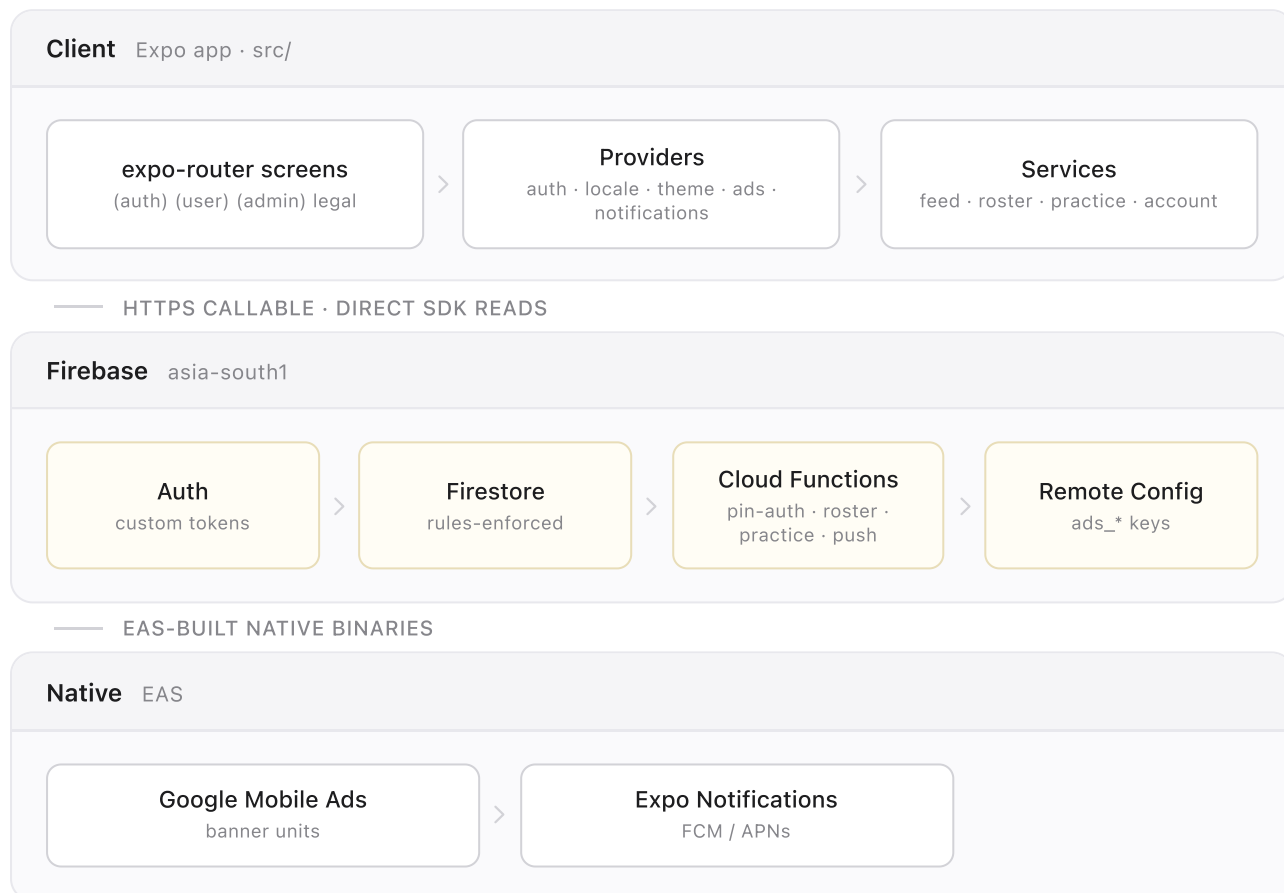
Member and admin experiences



The roster hands the member a join PIN, which closes the loop back into member sign-in.

Three tiers

The app is a thin client. Services own every Firebase conversation, providers own ads, notifications, locale, and theme, and screens compose the two.



Connections

- Services call Cloud Functions over HTTPS callables, and read Auth and Firestore directly.
- Providers read Remote Config, own the AdMob SDK, and own Expo Notifications.
- Cloud Functions write to Firestore and trigger push — the client never writes privileged data.

Guardrails

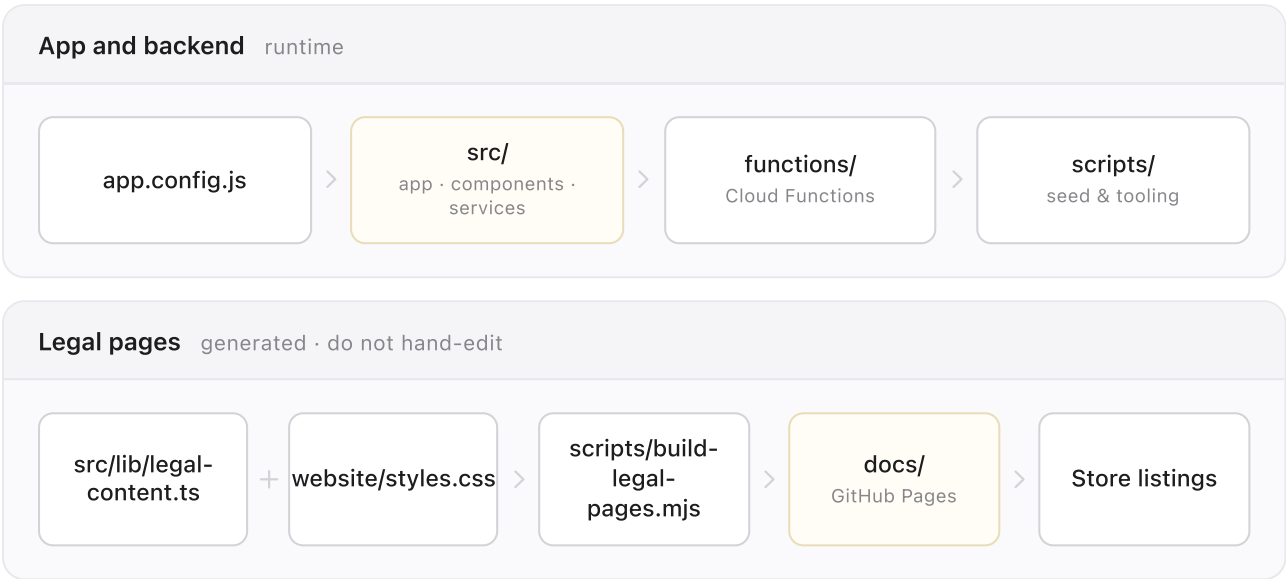
Enforced in the database

`firestore.rules` and `firestore.indexes.json` ship with the repo and deploy with the backend, so authorization does not depend on client code alone.

Privileged writes

PIN verification, roster edits, account deletion, and practice assignment all run server-side in `functions/src` behind callable authentication.

Where things live

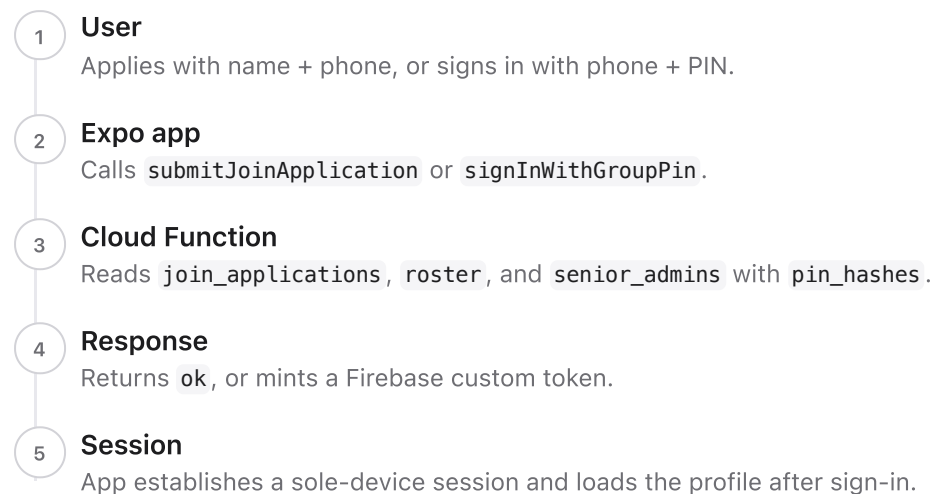


The same build runs in GitHub Actions, so the published site can never drift from the in-app legal copy.

Paths

PATH	ROLE
<code>src/app/</code>	Expo Router routes (auth, member tabs, admin, legal)
<code>src/services/</code>	Firestore / Functions client APIs
<code>src/providers/</code>	Auth, ads, locale, theme, notifications
<code>functions/src/</code>	Callable + scheduled backend
<code>src/lib/legal-content.ts</code>	Source of truth for in-app + public legal copy
<code>website/styles.css</code>	Styles for the generated Pages site
<code>docs/</code>	Generated Privacy / Terms HTML (<code>npm run build:legal-pages</code>)
<code>scripts/</code>	Seeds, icon prep, APK helpers (local PII stays gitignored)

Sign-in flow



Member

(user) home · seva · practice

Admin or Senior

(admin) dashboard

The three PINs

● Group join PIN

Issued by an admin from the roster. A new member signs in with phone + this PIN; admins can rotate it.

● Personal PIN

Set by an admin after their first login, required on every later sign-in.

● Senior admin PIN

Seeded through Admin SDK scripts only. Seniors are never created inside the app UI.

Access by role

ROLE	ACCESS
Applicant	Download the app; submit a join application. Signed out.
Senior admin	All groups; join applications; roster (members + admins); PINs; practice overview
Admin	Assigned groups; join applications; members; posts; practice
Member	Own group feed; today's practice; polls; alerts

What it is built on

● Client

- Expo SDK 57
- Expo Router
- React 19
- React Native
- NativeWind
- Reanimated

● Backend

- Firebase Auth
- Firestore
- Cloud Functions (asia-south1)
- Remote Config
- Scheduled cleanup & retention

● Monetization

- react-native-google-mobile-ads
- Banners only
- Remote Config kill switch

● Tooling

- TypeScript
- Vitest
- EAS Build
- Firebase CLI

At a glance

Firebase project `geeta-param-seva-6aa03`

Functions region `asia-south1`

Package license `UNLICENSED`

Node engine `^20.19.4 || ^22.13.0 || >=24.3.0`

Expo docs docs.expo.dev/versions/v54.0.0 — the versioned docs for this project's generation; see also the current SDK notes in `AGENTS.md`.

Quality checks

```
# .github/workflows/ci.yml runs these on every change
npm test
npm run typecheck
npm run functions:build
```

Local environment

Prerequisites

- Node.js `^20.19.4 || ^22.13.0 || >=24.3.0`
- Firebase project access + Application Default Credentials for seed scripts
- Expo / EAS CLI for store and device builds

Install

```
npm install
cp .env.example .env
cp functions/.env.example functions/.env
cp google-services.json.example google-services.json
cp GoogleService-Info.plist.example GoogleService-Info.plist
# Replace example Firebase / AdMob values with your local secrets
# (never commit them)
```

Local secret data (gitignored)

FILE	PURPOSE
<code>.env</code>	AdMob app IDs, feature flags
<code>google-services.json</code>	Firebase native config (Android)
<code>GoogleService-Info.plist</code>	Firebase native config (iOS)
<code>scripts/data/senior-admins.local.json</code>	Real senior phones, from <code>senior-admins.example.json</code>
<code>scripts/data/group-1-roster.json</code>	Real roster PII, from <code>group-1-roster.example.json</code>

Seed data

Senior admins

```
cp scripts/data/senior-admins.example.json scripts/data/senior-admins.local.json
# edit phones locally, then:
SENIOR_ADMIN_PIN=***** HRIDYA_SENIOR_ADMIN_PIN=***** node scripts/seed-senior-admin.cjs
```

QA & App Review accounts

App Review needs real, reachable logins. The seed script creates two valid Indian mobiles on the existing group 1 — no separate QA test group.

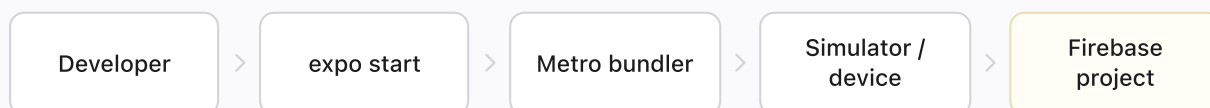
```
# Deploy functions first if member personal-PIN login is not yet live, then:
node scripts/seed-test-accounts.cjs
```

Admin	+919876500001	•	personal PIN 111111
Member	+919876500002	•	personal PIN 222222

Running the app

```
npx expo start
# or
npm run android
npm run ios
```

Local loop Metro



A development or EAS build is also required for the custom notification sound and for production AdMob — Expo Go is not enough.

Cloud Functions

```
npm run deploy:backend
# or, in batches when a cold start is too slow:
npm run deploy:backend:batch
```

Builds the functions bundle, then deploys functions plus Firestore rules and indexes to project `geeta-param-seva-6aa03`.

Important callables

submitJoinApplication

listJoinApplications

rejectJoinApplication

markJoinApplicationAdded

signInWithGroupPin

generateGroupJoinPin

setPersonalPin

roster upsert / deactivate

practice assignment / completion / reminder

wipeDailyFeed

deleteAccount

Placement

Member-only banners — no interstitial, no video. Placement is scroll-bound and kept away from primary taps.

SCREEN

BANNERS

Home

Up to 2 (slot 2 if height allows)

Seva

1 after the feed

Profile

1 at the bottom

Aarti · Gita · auth ·
notifications · admin

none

Enabling ads

- 1 Set `EXPO_PUBLIC_ADMOB_ANDROID_APP_ID` and `EXPO_PUBLIC_ADMOB_IOS_APP_ID` in `.env` or EAS secrets.
- 2 Publish Remote Config unit IDs plus `ads_enabled` via `npm run seed:ads-config:apply`.
- 3 Rebuild a native binary.

The kill switch

App IDs are baked in at build time, but the on/off state is not. Flipping `ads_enabled` in Remote Config removes every banner on the next app launch — no store release, no new binary.

What is never instrumented

● No ATT prompt

Banners are non-personalized, so App Tracking Transparency is never requested.

● No IDFA

The advertising identifier is neither read nor stored.

● No reading screens

Aarti, Gita, and every auth screen stay completely ad-free.

GitHub Pages

Do not hand-edit `docs/*.html`. Regenerate from source with `npm run build:legal-pages` — the HTML is a build artifact, not a source file.

```
npm run build:legal-pages
```

Pipeline: `src/lib/legal-content.ts` + `website/styles.css` → `scripts/build-legal-pages.mjs` → `docs/`. GitHub Actions runs the same build, so the site and the in-app copy stay identical.

Enable Pages once

Repository **Settings** → **Pages** → **Build and deployment** → **Source: GitHub Actions**. Then push to `main`, or run **Deploy legal pages** from Actions → `workflow_dispatch`.

Published pages

PAGE	URL
Home	https://sparshagrawal07.github.io/Geeta-param-seva/
Privacy Policy	https://sparshagrawal07.github.io/Geeta-param-seva/privacy-policy.html
Terms & Conditions	https://sparshagrawal07.github.io/Geeta-param-seva/terms.html

In-app legal screens mirror the same member-facing copy and link out to these URLs via `LEGAL_PRIVACY_URL` and `LEGAL_TERMS_URL` in `src/lib/legal-content.ts`.

Android

```
npm run build:android:apk      # EAS preview APK
npm run download:android:apk   # save → dist/Geeta-Param-Seva.apk
npm run build:android:aab      # Play Store AAB
npm run prepare:icons          # regenerate Expo icons from masters
```

All rights reserved

Copyright © Sparsh Agrawal. All Rights Reserved. This repository is **not** open source. No permission is granted to use, copy, modify, merge, publish, distribute, sublicense, or sell any part of this software without **prior written permission**.

Permissions	Sparshagrawaln@gmail.com
Support	geetaparamseva@gmail.com
License file	<code>LICENSE</code>
Package license	<code>UNLICENSED</code>



Geeta Param Seva

A moderated community app for Adhyay and Aarti practice,
Gita reading, and group seva coordination.

Support · geetaparamseva@gmail.com

Licensing · Sparshagrawal@gmail.com

Version 1.0.1 · Proprietary — All rights reserved

Generated from the project README.md